# Submission Worksheet

CLICK TO GRADE

https://learn.ethereallab.app/assignment/IT114-005-F2024/it114-module-3-number-guesser-4/grade/bna24

Course: IT114-005-F2024
Assigment: [IT114] Module 3 Number Guesser 4
Student: Bakr A. (bna24)

Submissions:

Submission Selection

1 Submission [submitted] 9/30/2024 9:04:09 PM

Instructions

COLLAPSE

Overview Video: https://youtu.be/ej6lWrg9XjE

1. Create the below branch name
2. Implement the NumberGuess4 example from the lesson/slides
    1. https://gist.github.com/MattToegel/aced06400c812f13ad030db9518b399f
    2. Add/commit the files as-is from the lesson material (this is the base template).
    3. Push the changes to the HW branch and create a pull request to keep open until this assignment is done
3. Pick two (2) of the following options to implement
    1. Display higher or lower as a hint after a wrong guess (only after a wrong guess that doesn't roll back the level)
    2. Implement anti-data tampering of the save file data (reject user direct edits)
    3. Add a difficulty selector that adjusts the max strikes per level (i.e., "easy" 10 strikes, "medium" 5 strikes, "hard" 3 strikes)
    4. Display a cold, warm, hot indicator based on how close to the correct value the guess is (example, 10 numbers away is cold, 5 numbers away is warm, 2 numbers away is hot; adjust these per your preference) Only display this when the wrong guess doesn't roll back the level
    5. Add a hint command that can be used once per level and only after 2 strikes have been used that reduces the range around the correct number (i.e., number is 5 and range is initially 1-15, new range could be 3-8 as a hint)
    6. Implement separate save files based on a "What's your name?" prompt at the start of the game (each person gets their own save file based on user's name)
4. Fill in the below deliverables
5. Save changes and export PDF

6. Git add/commit/push your changes to the HW branch
7. Create a pull request to main (if not done so before)
8. Complete the pull request (don't forget to locally checkout main and pull changes to prep for future work)
9. Upload the same PDF to Canvas

**Branch name:** M3-NumberGuesser-4

Group

100%

Group: Implementation 1
Tasks: 1
Points: 4

COLLAPSE

Task

100%

Group: Implementation 1
Task #1: Implementation Evidence
Weight: ~100%
Points: ~4.00

COLLAPSE

Details:

Code screenshots must have ucid/date shown as a comment in the code.

Explanations must be your own words describing the logic and how the solution code solves the problem.

Columns: 1

Sub-Task

100%

Group: Implementation 1
Task #1: Implementation Evidence
Sub Task #1: Mention which option you picked and how you solved it

# Task Response Prompt

*Explain the logic of how you solved/implemented the chosen option (concrete details). Explain how the code works, don't just paste code snippets*

Response:

I choose Implementation 1, "Display higher or lower as a hint after a wrong guess (only after a wrong guess that doesn't roll back the level)". I went into the method "processGuess" because that is where the logic that I need would happen. After the code runs and reaches the point where it says "That's wrong", I implimented an if statement where it takes the guess and compares it against the number. If the guess is lower than the number it goes to the else if statement and I wrote System.out.println("hint: answer is higher"). And if it was higher, System.out.println("hint: answer is lower").

Sub-Task

100%

Group: Implementation 1

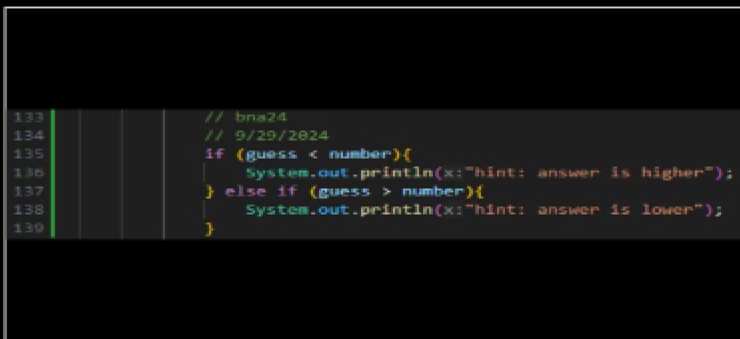
Task #1: Implementation Evidence

Sub Task #2: Add screenshots of the coded solution (ucid/date must be visible)

## Task Screenshots

Gallery Style: 2 Columns

4 2 1




Code for Option #1

**Caption(s) (required)** ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task

100%

Group: Implementation 1

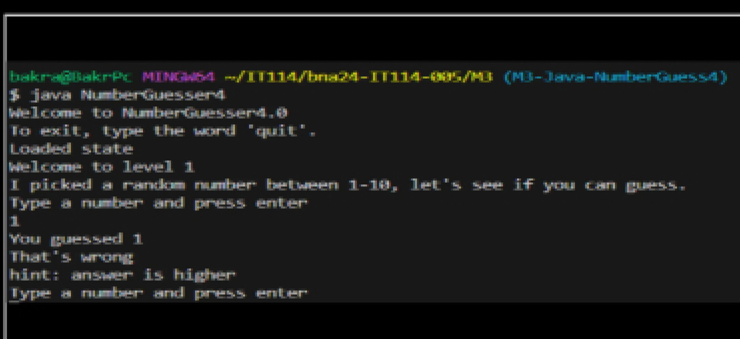
Task #1: Implementation Evidence

Sub Task #3: Show implementation working by running the program

## Task Screenshots

Gallery Style: 2 Columns

4 2 1




Terminal Output For Problem #1

**Caption(s) (required)** ✓

Caption Hint: *Describe/highlight what's being shown*

End of Task 1

End of Group: Implementation 1

Task Status: 1/1

Group

100%

Group: Implementation 2
Tasks: 1
Points: 4

COLLAPSE

Task

100%

Group: Implementation 2
Task #1: Implementation Evidence
Weight: ~100%
Points: ~4.00

COLLAPSE

Details:
Code screenshots must have ucid/date shown as a comment in the code.

Explanations must be your own words describing the logic and how the solution code solves the problem.

Columns: 1

Sub-Task

100%

Group: Implementation 2
Task #1: Implementation Evidence
Sub Task #1: Mention which option you picked and how you solved it

## Task Response Prompt

*Explain the logic of how you solved/implemented the chosen option (concrete details). Explain how the code works, don't just paste code snippets*

Response:

I chose option 3, The select difficulty feature. this feature allows the player to choose how challenging they want the game to be at the start. The game gives the player three options: easy, medium, or hard. Each difficulty level controls how many strikes the player is allowed before they lose the level. When the game starts, the player is asked to type either easy, medium, or hard. Bassed on the response, the game adjusts the number of strikes from 10 (easy), 5 (medium), and 3 (hard). if the player types something else it will pick medium as the default value.

Sub-Task

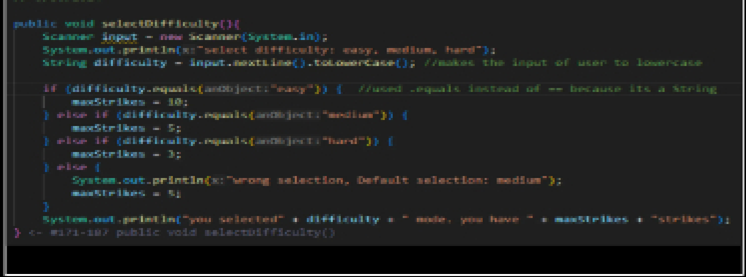
100%

Group: Implementation 2
Task #1: Implementation Evidence
Sub Task #2: Add screenshots of the coded solution (ucid/date must be visible)

## Task Screenshots

Gallery Style: 2 Columns

4 2 1



Implementation for #3 code

**Caption(s) (required)** ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task

100%

Group: Implementation 2

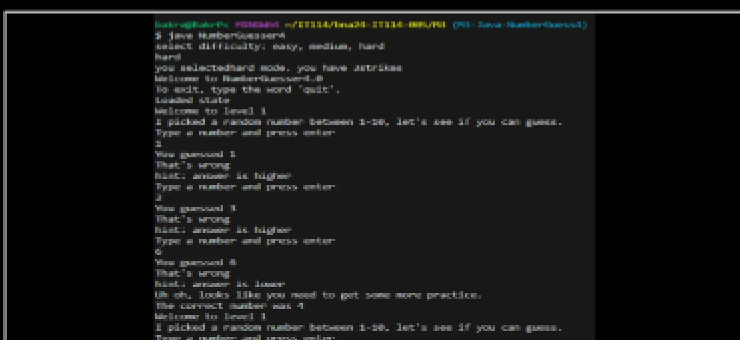
Task #1: Implementation Evidence

Sub Task #3: Show implementation working by running the program

## Task Screenshots

Gallery Style: 2 Columns

4 2 1



Terminal output for #3

**Caption(s) (required)** ✓

Caption Hint: *Describe/highlight what's being shown*

End of Task 1

End of Group: Implementation 2

Task Status: 1/1

Group

100%

Group: Misc

Tasks: 3

Points: 2

COLLAPSE

Task

100%

Group: Misc

Task #1: Reflection

Weight: ~33%

Points: ~0.67

COLLAPSE

Sub-Task

100%

Group: Misc
Task #1: Reflection
Sub Task #1: Learn anything new? Face any challenges? How did you overcome any issues?

## Task Response Prompt

*Provide at least a few logical sentences*

Response:

I learned how to read and edit already existing code which is different from making your own from scratch. I found myself reading to code to try to understand what is going on rather than coding.

End of Task 1

Task

100%

Group: Misc
Task #2: Pull Request URL
Weight: ~33%
Points: ~0.67

COLLAPSE

Details:
URL should end with /pull/# where the # is the actual pull request number.

## Task URLs

URL #1
https://github.com/BakrAbushaar/bna24-IT114-005/pull/7

URL
https://github.com/BakrAbushaar/bna24-IT114-0

End of Task 2

Task

100%

Group: Misc
Task #3: Waka Time (or related) Screenshot
Weight: ~33%
Points: ~0.67

COLLAPSE

Checklist

*The checkboxes are for your own tracking

| # | Details |
| --- | --- |

| # | Details |
| --- | --- |
| ☐ #1 | Screenshot clearly shows what files/project were being worked on (the duration of time doesn't correlated with the grade for this item) |

## Task Screenshots

Gallery Style: 2 Columns

4 2 1


WakaTime For NumberGuesser4.java

End of Task 3

End of Group: Misc
Task Status: 3/3

End of Assignment